Department of Computer Science and Engineering,
School of Technology,
Pandit Deendayal Energy University (PDEU)
Design Analysis and Algorithm Laboratory (24CS210P)

Lab Assignment 1

Name: Dax Patel
Roll no: 24BCP269

Title: Understanding basic of Sorting algorithms.

Objective: These lab experiments cover fundamental aspects of sorting algorith

CODE:

```c
#include<stdio.h>
#include<time.h>

int counter=0;
int swaps=0;

void bubble_sort(int arr[] , int size)
{
    int start_time = clock();
    for(int i=0 ; i < size-1 ; i++)
    {
        for(int j=0 ; j<size-i-1 ;j++)
        {
            counter++;
            if(arr[j]>arr[j+1])
            {
                swaps++;
                int temp = arr[j];
                arr[j]=arr[j+1];
                arr[j+1]=temp;
            }
        }
    }
    int end_time=clock();
    printf("Bubble sorted array is :");
    for(int i=0 ; i<size ;i++)
    {
        printf("%d ",arr[i]);
    }
    printf("The comparisions for the Bubble sort is :%d \n",counter);

    printf("The total swaps that are happens is : %d \n",swaps);

    printf("Time taken for the Code to exicute is : %ds \n",(end_time-start_time)*1000/60);

}

void selection_sort(int arr[] ,int size)
{
    for(int i=0 ; i<size-1;i++)
    {
        int minindex = i;
        for(int j=i+1 ; j<size ; j++)
        {
            if(arr[j]<arr[minindex])
            {
               minindex=j;
            }
        }

        int temp = arr[minindex];
                arr[minindex]=arr[i];
                arr[i]=temp;
    }
```

```
    printf("selection sorted array is :");
    for(int i=0 ; i<size ;i++)
    {
        printf("%d ",arr[i]);
    }
}

void insertionSort(int arr[],int size) {
    for (int i = 1; i < size; i++) {

        int key = arr[i];
        int j = i - 1;


        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }

        arr[j + 1] = key;
    }

    printf("Insertion sorted array is :");
    for(int i=0 ; i<size ;i++)
    {
        printf("%d ",arr[i]);
    }
}

void swap(int* num1, int* num2)
{
    int temp = *num1;
    *num1 = *num2;
    *num2 = temp;
}

void quick_sort(int arr[], int start, int end)
{
    if (start >= end)
        return;

    int i = start - 1;
    int j = start;
    int pivot = end;

    while (j < pivot)
    {
        if (arr[j] < arr[pivot])
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
        j++;
    }

    i++;
```

```
    swap(&arr[i], &arr[pivot]);

    quick_sort(arr, start, i - 1);
    quick_sort(arr, i + 1, end);
}

void merge1(int arr[], int start, int mid, int end) {
    int len1 = mid - start + 1;
    int len2 = end - mid;

    int left[len1], right[len2];

    for (int i = 0; i < len1; i++)
        left[i] = arr[start + i];

    for (int j = 0; j < len2; j++)
        right[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = start;

    // dono array ko merge karr denge
    while (i < len1 && j < len2) {
        if (left[i] < right[j])
            arr[k++] = left[i++];
        else
            arr[k++] = right[j++];
    }

    //copy karr denge
    while (i < len1)
        arr[k++] = left[i++];

    while (j < len2)
        arr[k++] = right[j++];
}

void merge_sort(int arr[], int start, int end) {
    if (start >= end)
        return;

    int mid = start + (end - start) / 2;

    //do array bana diye equl devide karke
    merge_sort(arr, start, mid);
    merge_sort(arr, mid + 1, end);

    //dono ko merge kar diya;
    merge1(arr, start, mid, end);
}

int  main()
{

    int size;

    printf("Enter the size of the array: ");
```

```
    scanf("%d",&size);

    int original_arr[size];
    for(int i=0 ; i<size ;i++)
    {
        printf("Enter the number: ");
        scanf("%d",&original_arr[i]);
    }

    while (1) {
        int arr[size];
        for(int i=0 ; i<size ;i++)
        {
            arr[i] = original_arr[i];
        }

        printf("\nOriginal array: ");
        for(int i=0 ; i<size ;i++)
        {
            printf("%d ", arr[i]);
        }

        printf("\n\n");

        int choice;
        printf("Choose a sorting algorithm:\n");
        printf("1. Bubble Sort\n");
        printf("2. Selection Sort\n");
        printf("3. Insertion Sort\n");
        printf("4. Quick Sort\n");
        printf("5. Merge Sort\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        printf("\n");

        if (choice == 6) {
            printf("Exiting...\n");
            break;
        }

        switch (choice) {
            case 1:
                bubble_sort(arr, size);
                break;
            case 2:
                selection_sort(arr, size);
                break;
            case 3:
                insertionSort(arr, size);
                break;
            case 4:
                quick_sort(arr, 0, size - 1);
                printf("Quick sorted array is :");
                for(int i=0 ; i<size ;i++) {
```

```
                    printf("%d ",arr[i]);
                }
                break;
            case 5:
                merge_sort(arr, 0, size - 1);
                printf("Merge sorted array is :");
                for(int i=0 ; i<size ;i++) {
                    printf("%d ",arr[i]);
                }
                break;
            default:
                printf("Invalid choice!\n");
        }

        printf("\n");

        if (choice >= 1 && choice <= 3) {
            printf("All of these sorting algorithms run in O(N^2) time complexity.\n");
        } else if (choice == 4 || choice == 5) {
            printf("This algorithm sorts in O(N log N) time complexity.\n");
        }

        printf("-------------------------------------------------\n");
    }

    return 0;
}
```

OUTPUT:

```
(base) daxpatel@Daxs-MacBook-Air LAB1(Sorting_arr) % cc sort.c -o sort && ./sort

Enter the size of the array: 4
Enter the number: 23
Enter the number: 54
Enter the number: 12
Enter the number: 87

Original array: 23 54 12 87

Choose a sorting algorithm:
1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort
6. Exit
Enter your choice: 1

Bubble sorted array is :12 23 54 87 The comparisions for the Bubble sort is :6
The total swaps that are happens is : 2
Time taken for the Code to exicute is : 50s

All of these sorting algorithms run in O(N^2) time complexity.
------------------------------------------------

Original array: 23 54 12 87

Choose a sorting algorithm:
1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort
6. Exit
Enter your choice: 2

selection sorted array is :12 23 54 87
All of these sorting algorithms run in O(N^2) time complexity.
------------------------------------------------

Original array: 23 54 12 87

Choose a sorting algorithm:
1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort
6. Exit
Enter your choice: 3

Insertion sorted array is :12 23 54 87
All of these sorting algorithms run in O(N^2) time complexity.
------------------------------------------------
```

```
Choose a sorting algorithm:
1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort
6. Exit
Enter your choice: 2

selection sorted array is :12 23 54 87
All of these sorting algorithms run in O(N^2) time complexity.
-------------------------------------------------

Original array: 23 54 12 87

Choose a sorting algorithm:
1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort
6. Exit
Enter your choice: 3

Insertion sorted array is :12 23 54 87
All of these sorting algorithms run in O(N^2) time complexity.
-------------------------------------------------

Original array: 23 54 12 87

Choose a sorting algorithm:
1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort
6. Exit
Enter your choice: 4

Quick sorted array is :12 23 54 87
This algorithm sorts in O(N log N) time complexity.
-------------------------------------------------
```

```
Original array: 23 54 12 87

Choose a sorting algorithm:
1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort
6. Exit
Enter your choice: 5

Merge sorted array is :12 23 54 87
This algorithm sorts in O(N log N) time complexity.
------------------------------------------------

Original array: 23 54 12 87

Choose a sorting algorithm:
1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort
6. Exit
Enter your choice: 6

Exiting...
```

Department of Computer Science and Engineering,
School of Technology,
Pandit Deendayal Energy University (PDEU)
Design Analysis and Algorithm Laboratory (24CS210P)

Lab Assignment 2

Name: Dax Patel
Roll no: 24BCP269

Title: Understanding basic of Searching algorithms.

Objective: These lab experiments cover fundamental aspects of searching algorithm

CODE:

```cpp
#include <algorithm>
#include <chrono>
#include <iostream>
#include <vector>

using namespace std;
using namespace std::chrono;

int binarysearch(const vector<int> &arr, int target) {
  int start = 0;
  int end = arr.size() - 1;

  while (start <= end) {
    int mid = start + (end - start) / 2;

    if (arr[mid] == target) {
      return mid;
    } else if (arr[mid] > target) {
      end = mid - 1;
    } else {
      start = mid + 1;
    }
  }
  return -1;
}

int linersearch(const vector<int> &arr, int target) {
  for (int i = 0; i < arr.size(); i++) {
    if (arr[i] == target) {
      return i;
    }
  }
  return -1;
}

int main() {
  int size;
  cout << "Enter the size of the array: ";
  cin >> size;

  vector<int> arr(size);
  for (int i = 0; i < size; i++) {
    cout << "Enter element " << i + 1 << ": ";
    cin >> arr[i];
  }

  vector<int> sorted_arr = arr;
  sort(sorted_arr.begin(), sorted_arr.end());

  cout << "\nOriginal array: ";
  for (int x : arr) {
    cout << x << " ";
  }
```

```
cout << "\nSorted array (for Binary Search): ";
for (int x : sorted_arr) {
  cout << x << " ";
}
cout << "\n\n";

while (true) {
  int choice;
  cout << "Choose a search algorithm:\n";
  cout << "1. Linear Search\n";
  cout << "2. Binary Search\n";
  cout << "3. Exit\n";
  cout << "Enter your choice: ";
  cin >> choice;

  if (choice == 3) {
    cout << "Exiting...\n";
    break;
  }

  int target;
  if (choice == 1 || choice == 2) {
    cout << "Enter the target element to search: ";
    cin >> target;
  }

  auto start_time = high_resolution_clock::now();
  auto end_time = high_resolution_clock::now();
  int result = -1;

  switch (choice) {
  case 1:
    start_time = high_resolution_clock::now();
    result = linersearch(arr, target);
    end_time = high_resolution_clock::now();
    break;
  case 2:
    start_time = high_resolution_clock::now();
    result = binarysearch(sorted_arr, target);
    end_time = high_resolution_clock::now();
    break;
  default:
    cout << "Invalid choice!\n\n";
    continue;
  }

  if (result != -1) {
    if (choice == 1) {
      cout << "Element found at index " << result << " in Original array.\n";
    } else {
      cout << "Element found at index " << result << " in Sorted array.\n";
    }
  } else {
    cout << "Element not found.\n";
  }
```

```
    auto duration = duration_cast<nanoseconds>(end_time - start_time);
    cout << "Time taken: " << duration.count() << " nanoseconds.\n\n";
    cout << "-------------------------------------------------\n";
  }

  return 0;
}
```

OUTPUT:

```
(base) daxpatel@Daxs-MacBook-Air Lab2(search) % g++ -std=c++17 binarysearch.cpp -o binarysearch && ./binarysearch

Invalid choice!

Choose a search algorithm:
1. Linear Search
2. Binary Search
3. Exit
Enter your choice: 1
Enter the target element to search: 32
Element found at index 0 in Original array.
Time taken: 1084 nanoseconds.

-------------------------------------------------
Choose a search algorithm:
1. Linear Search
2. Binary Search
3. Exit
Enter your choice: 2
Enter the target element to search: 32
Element found at index 1 in Sorted array.
Time taken: 1459 nanoseconds.

-------------------------------------------------
Choose a search algorithm:
1. Linear Search
2. Binary Search
3. Exit
Enter your choice: 3
Exiting...
(base) daxpatel@Daxs-MacBook-Air Lab2(search) %
```

Department of Computer Science and Engineering,
School of Technology,
Pandit Deendayal Energy University (PDEU)
Design Analysis and Algorithm Laboratory
(24CS210P)

Lab Assignment 3

Name: Dax Patel
Roll no: 24BCP269

Title: Understanding basic of Linked Lists.

Objective: These lab experiments cover fundamental aspects of linked lists

Task 1: Use singly linked lists to implement integers of unlimited size. Each node of the list should store one digit of the integer. Implement addition, subtraction, multiplication, and exponentiation operations. Limit exponents to be positive integers.
What is the asymptotic running time for each of your operations, expressed in terms of the number of digits for the two operands of each function?

CODE:

```
#include <iostream>
#include <string>

using namespace std;

class Node {
public:
  int data;
  Node *next;
  Node(int d) : data(d), next(nullptr) {}
};

class BigInt {
public:
  Node *head;

  BigInt() : head(nullptr) {}

  BigInt(string s) {
    head = nullptr;
    Node *tail = nullptr;

    for (int i = s.length() - 1; i >= 0; i--) {
      if (s[i] < '0' || s[i] > '9')
        continue;
      Node *nn = new Node(s[i] - '0');
      if (!head) {
        head = nn;
        tail = nn;
      } else {
        tail->next = nn;
        tail = nn;
      }
    }
    if (!head)
      head = new Node(0);
  }

  BigInt(const BigInt &other) {
    head = nullptr;
    Node *curr = other.head;
    Node *tail = nullptr;
    while (curr) {
      Node *nn = new Node(curr->data);
      if (!head) {
        head = nn;
```

```
      tail = nn;
    } else {
      tail->next = nn;
      tail = nn;
    }
    curr = curr->next;
  }
}

~BigInt() {
  Node *curr = head;
  while (curr) {
    Node *temp = curr;
    curr = curr->next;
    delete temp;
  }
}

BigInt &operator=(const BigInt &other) {
  if (this == &other)
    return *this;

  Node *curr = head;
  while (curr) {
    Node *temp = curr;
    curr = curr->next;
    delete temp;
  }
  head = nullptr;

  curr = other.head;
  Node *tail = nullptr;
  while (curr) {
    Node *nn = new Node(curr->data);
    if (!head) {
      head = nn;
      tail = nn;
    } else {
      tail->next = nn;
      tail = nn;
    }
    curr = curr->next;
  }
  return *this;
}

void trim() {

  if (!head)
    return;

  Node *prev = nullptr;
  Node *curr = head;
  while (curr) {
    Node *nxt = curr->next;
    curr->next = prev;
```

```
    prev = curr;
    curr = nxt;
  }
  head = prev;

  while (head && head->data == 0 && head->next) {
    Node *temp = head;
    head = head->next;
    delete temp;
  }

  prev = nullptr;
  curr = head;
  while (curr) {
    Node *nxt = curr->next;
    curr->next = prev;
    prev = curr;
    curr = nxt;
  }
  head = prev;
}

BigInt add(const BigInt &other) const {
  BigInt res;
  Node *t1 = head;
  Node *t2 = other.head;
  Node *tail = nullptr;
  int carry = 0;

  while (t1 || t2 || carry) {
    int sum = carry;
    if (t1) {
      sum += t1->data;
      t1 = t1->next;
    }
    if (t2) {
      sum += t2->data;
      t2 = t2->next;
    }
    carry = sum / 10;

    Node *nn = new Node(sum % 10);
    if (!res.head) {
      res.head = nn;
      tail = nn;
    } else {
      tail->next = nn;
      tail = nn;
    }
  }
  return res;
}

BigInt sub(const BigInt &other) const {
  BigInt res;
  Node *t1 = head;
```

```
  Node *t2 = other.head;
  Node *tail = nullptr;
  int borrow = 0;

  while (t1) {
    int diff = t1->data - borrow;
    if (t2) {
      diff -= t2->data;
      t2 = t2->next;
    }
    if (diff < 0) {
      diff += 10;
      borrow = 1;
    } else {
      borrow = 0;
    }
    Node *nn = new Node(diff);
    if (!res.head) {
      res.head = nn;
      tail = nn;
    } else {
      tail->next = nn;
      tail = nn;
    }
    t1 = t1->next;
  }
  res.trim();
  return res;
}

BigInt mul(const BigInt &other) const {
  BigInt result("0");
  Node *t2 = other.head;
  int shift = 0;

  while (t2) {
    if (t2->data != 0) {
      BigInt temp;
      Node *tail = nullptr;
      for (int i = 0; i < shift; i++) {
        Node *nn = new Node(0);
        if (!temp.head) {
          temp.head = nn;
          tail = nn;
        } else {
          tail->next = nn;
          tail = nn;
        }
      }

      Node *t1 = head;
      int carry = 0;
      while (t1 || carry) {
        int prod = carry;
        if (t1) {
          prod += t1->data * t2->data;
```

```
          t1 = t1->next;
        }
        carry = prod / 10;
        Node *nn = new Node(prod % 10);
        if (!temp.head) {
          temp.head = nn;
          tail = nn;
        } else {
          tail->next = nn;
          tail = nn;
        }
      }
      result = result.add(temp);
    }
    shift++;
    t2 = t2->next;
  }
  result.trim();
  return result;
}

BigInt exp(long long e) const {
  if (e == 0)
    return BigInt("1");
  if (e == 1)
    return *this;

  BigInt base = *this;
  BigInt res("1");

  while (e > 0) {
    if (e % 2 == 1) {
      res = res.mul(base);
    }
    base = base.mul(base);
    e /= 2;
  }
  return res;
}

void print() const {
  if (!head) {
    cout << "0";
    return;
  }
  string s = "";
  Node *curr = head;
  while (curr) {
    s += to_string(curr->data);
    curr = curr->next;
  }
  for (int i = s.length() - 1; i >= 0; i--) {
    cout << s[i];
  }
}
```

```
  long long toLongLong() const {
    long long res = 0;
    long long multiplier = 1;
    Node *curr = head;
    while (curr) {
      res += curr->data * multiplier;
      multiplier *= 10;
      curr = curr->next;
    }
    return res;
  }
};

int main() {
  string str1, str2;
  cout << "Enter the first infinite precision integer (No signs): ";
  cin >> str1;
  cout << "Enter the second infinite precision integer (No signs): ";
  cin >> str2;

  BigInt num1(str1);
  BigInt num2(str2);

  while (true) {
    cout << "\nOperations:\n";
    cout << "1. Addition (num1 + num2)\n";
    cout << "2. Subtraction (num1 - num2, assumes num1 >= num2)\n";
    cout << "3. Multiplication (num1 * num2)\n";
    cout << "4. Exponentiation (num1 ^ num2)\n";
    cout << "5. Exit\n";

    int choice;
    cout << "Enter choice: ";
    cin >> choice;

    if (choice == 5) {
      cout << "Exiting...\n";
      break;
    }

    cout << "\nResult: ";
    switch (choice) {
    case 1: {
      BigInt res = num1.add(num2);
      res.print();
      break;
    }
    case 2: {
      BigInt res = num1.sub(num2);
      res.print();
      break;
    }
    case 3: {
      BigInt res = num1.mul(num2);
      res.print();
      break;
```

```
    }
    case 4: {
      long long expValue = num2.toLongLong();
      BigInt res = num1.exp(expValue);
      res.print();
      break;
    }
    default:
      cout << "Invalid choice.";
    }
    cout << "\n";
  }

  return 0;
}
```

OUTPUT:

```
(base) daxpatel@Daxs-MacBook-Air Linkedlist % cd /Users/daxpatel/Desktop/DSA/Linkedlist
g++ -std=c++17 singleLL.cpp -o singleLL
./singleLL

Enter the first infinite precision integer (No signs): 4
Enter the second infinite precision integer (No signs): 6

Operations:
1. Addition (num1 + num2)
2. Subtraction (num1 - num2, assumes num1 >= num2)
3. Multiplication (num1 * num2)
4. Exponentiation (num1 ^ num2)
5. Exit
Enter choice: 1

Result: 10

Operations:
1. Addition (num1 + num2)
2. Subtraction (num1 - num2, assumes num1 >= num2)
3. Multiplication (num1 * num2)
4. Exponentiation (num1 ^ num2)
5. Exit
Enter choice: 2

Result: 8

Operations:
1. Addition (num1 + num2)
2. Subtraction (num1 - num2, assumes num1 >= num2)
3. Multiplication (num1 * num2)
4. Exponentiation (num1 ^ num2)
5. Exit
Enter choice: 3

Result: 24

Operations:
1. Addition (num1 + num2)
2. Subtraction (num1 - num2, assumes num1 >= num2)
3. Multiplication (num1 * num2)
4. Exponentiation (num1 ^ num2)
5. Exit
Enter choice: 4

Result: 4096

Operations:
1. Addition (num1 + num2)
2. Subtraction (num1 - num2, assumes num1 >= num2)
3. Multiplication (num1 * num2)
4. Exponentiation (num1 ^ num2)
5. Exit
Enter choice: 5
```

Department of Computer Science and Engineering,
School of Technology,
Pandit Deendayal Energy University (PDEU)
Design Analysis and Algorithm Laboratory (24CS210P)

Lab Assignment 4

Name:Dax Patel
Roll no: 24BCP269

Title: Understanding difference between Arrays and Linked Lists.

Objective: These lab experiments cover the time for different operation in arrays and linked lists

Task 1 Implement a city database using unordered lists. Each database record contains the name of the city (a string of arbitrary length) and the coordinates of the city expressed as integer x and y coordinates. Your program should allow following functionalities:
a) Insert a record,
b) Delete a record by name or coordinate,
c) Search a record by name or coordinate.
d) Print all records within a given distance of a specified point.

Implement the database using an array-based list implementation, and then a linked list implementation. Perform following analysis:
a) Collect running time statistics for each operation in both implementations.
b) What are your conclusions about the relative advantages and disadvantages of the two implementations?
c) Would storing records on the list in alphabetical order by city name speed any of the operations?
d) Would keeping the list in alphabetical order slow any of the operations?

CODE:

```
#include <chrono>
#include <cmath>
#include <iostream>
#include <string>
#include <vector>

using namespace std;
using namespace std::chrono;

struct City {
  string name;
  int x;
  int y;

  City() : name(""), x(0), y(0) {}
  City(string n, int x_coord, int y_coord) : name(n), x(x_coord), y(y_coord) {}
};

// ARRAY-BASED LIST IMPLEMENTATION
class ArrayListDB {
private:
  City *db;
  int capacity;
  int size;

  void resize() {
    capacity *= 2;
    City *new_db = new City[capacity];
    for (int i = 0; i < size; ++i) {
      new_db[i] = db[i];
    }
    delete[] db;
    db = new_db;
  }

public:
  ArrayListDB(int cap = 10) : capacity(cap), size(0) {
    db = new City[capacity];
```

```
}

~ArrayListDB() { delete[] db; }

void insertRecord(string name, int x, int y) {
  if (size == capacity)
    resize();
  db[size++] = City(name, x, y);
}

void deleteByName(string name) {
  for (int i = 0; i < size; ++i) {
    if (db[i].name == name) {
      for (int j = i; j < size - 1; ++j) {
        db[j] = db[j + 1];
      }
      size--;
      i--;
    }
  }
}

void deleteByCoordinate(int x, int y) {
  for (int i = 0; i < size; ++i) {
    if (db[i].x == x && db[i].y == y) {
      for (int j = i; j < size - 1; ++j) {
        db[j] = db[j + 1];
      }
      size--;
      i--;
    }
  }
}

void searchByName(string name) {
  bool found = false;
  for (int i = 0; i < size; ++i) {
    if (db[i].name == name) {
      cout << "Found: " << db[i].name << " at (" << db[i].x << ", " << db[i].y
           << ")\n";
      found = true;
    }
  }
  if (!found)
    cout << "City '" << name << "' not found.\n";
}

void searchByCoordinate(int x, int y) {
  bool found = false;
  for (int i = 0; i < size; ++i) {
    if (db[i].x == x && db[i].y == y) {
      cout << "Found: " << db[i].name << " at (" << db[i].x << ", " << db[i].y
           << ")\n";
      found = true;
    }
  }
```

```
    if (!found)
      cout << "No city found at (" << x << ", " << y << ").\n";
  }

  void printWithinDistance(int px, int py, double distance) {
    cout << "Cities within distance " << distance << " from (" << px << ", "
         << py << "):\n";
    bool found = false;
    for (int i = 0; i < size; ++i) {
      double dist = sqrt(pow(db[i].x - px, 2) + pow(db[i].y - py, 2));
      if (dist <= distance) {
        cout << "- " << db[i].name << " at (" << db[i].x << ", " << db[i].y
             << ") [Distance: " << dist << "]\n";
        found = true;
      }
    }
    if (!found)
      cout << "None found.\n";
  }

  void printAll() {
    for (int i = 0; i < size; i++) {
      cout << "- " << db[i].name << " at (" << db[i].x << ", " << db[i].y
           << ")\n";
    }
  }
};

// LINKED LIST IMPLEMENTATION

struct Node {
  City data;
  Node *next;
  Node(string n, int x, int y) : data(n, x, y), next(nullptr) {}
};

class LinkedListDB {
private:
  Node *head;

public:
  LinkedListDB() : head(nullptr) {}

  ~LinkedListDB() {
    Node *curr = head;
    while (curr) {
      Node *temp = curr;
      curr = curr->next;
      delete temp;
    }
  }

  void insertRecord(string name, int x, int y) {
    Node *newNode = new Node(name, x, y);
    if (!head) {
      head = newNode;
```

```
  } else {
    Node *curr = head;
    while (curr->next) {
      curr = curr->next;
    }
    curr->next = newNode;
  }
}

void deleteByName(string name) {
  while (head && head->data.name == name) {
    Node *temp = head;
    head = head->next;
    delete temp;
  }

  Node *curr = head;
  while (curr && curr->next) {
    if (curr->next->data.name == name) {
      Node *temp = curr->next;
      curr->next = temp->next;
      delete temp;
    } else {
      curr = curr->next;
    }
  }
}

void deleteByCoordinate(int x, int y) {
  while (head && head->data.x == x && head->data.y == y) {
    Node *temp = head;
    head = head->next;
    delete temp;
  }

  Node *curr = head;
  while (curr && curr->next) {
    if (curr->next->data.x == x && curr->next->data.y == y) {
      Node *temp = curr->next;
      curr->next = temp->next;
      delete temp;
    } else {
      curr = curr->next;
    }
  }
}

void searchByName(string name) {
  Node *curr = head;
  bool found = false;
  while (curr) {
    if (curr->data.name == name) {
      cout << "Found: " << curr->data.name << " at (" << curr->data.x << ", "
           << curr->data.y << ")\n";
      found = true;
    }
```

```
      curr = curr->next;
    }
    if (!found)
      cout << "City '" << name << "' not found.\n";
  }

  void searchByCoordinate(int x, int y) {
    Node *curr = head;
    bool found = false;
    while (curr) {
      if (curr->data.x == x && curr->data.y == y) {
        cout << "Found: " << curr->data.name << " at (" << curr->data.x << ", "
             << curr->data.y << ")\n";
        found = true;
      }
      curr = curr->next;
    }
    if (!found)
      cout << "No city found at (" << x << ", " << y << ").\n";
  }

  void printWithinDistance(int px, int py, double distance) {
    cout << "Cities within distance " << distance << " from (" << px << ", "
         << py << "):\n";
    Node *curr = head;
    bool found = false;
    while (curr) {
      double dist = sqrt(pow(curr->data.x - px, 2) + pow(curr->data.y - py, 2));
      if (dist <= distance) {
        cout << "- " << curr->data.name << " at (" << curr->data.x << ", "
             << curr->data.y << ") [Distance: " << dist << "]\n";
        found = true;
      }
      curr = curr->next;
    }
    if (!found)
      cout << "None found.\n";
  }

  void printAll() {
    Node *curr = head;
    while (curr) {
      cout << "- " << curr->data.name << " at (" << curr->data.x << ", "
           << curr->data.y << ")\n";
      curr = curr->next;
    }
  }
};

int main() {
  ArrayListDB arrayDB;
  LinkedListDB linkedDB;

  while (true) {
    cout << "\n========= CITY DATABASE MENU =========\n";
    cout << "1. Insert Data\n";
```

```
    cout << "2. Print List (Array DB)\n";
    cout << "3. Print List (Linked List DB)\n";
    cout << "4. Search by Name\n";
    cout << "5. Search by Coordinate\n";
    cout << "6. Delete by Name\n";
    cout << "7. Delete by Coordinate\n";
    cout << "8. Print within Distance\n";
    cout << "9. Exit\n";
    cout << "Enter choice: ";

    int choice;
    cin >> choice;

    if (choice == 9) {
      cout << "Exiting...\n";
      break;
    }

    switch (choice) {
    case 1: {
      string name;
      int x, y;
      cout << "Enter Name: ";
      cin >> name;
      cout << "Enter X: ";
      cin >> x;
      cout << "Enter Y: ";
      cin >> y;
      arrayDB.insertRecord(name, x, y);
      linkedDB.insertRecord(name, x, y);
      cout << "Data inserted into both databases.\n";
      break;
    }
    case 2:
      cout << "--- Array DB ---\n";
      arrayDB.printAll();
      break;
    case 3:
      cout << "--- Linked List DB ---\n";
      linkedDB.printAll();
      break;
    case 4: {
      string name;
      cout << "Enter name to search: ";
      cin >> name;
      cout << "Array DB: ";
      arrayDB.searchByName(name);
      cout << "Linked List DB: ";
      linkedDB.searchByName(name);
      break;
    }
    case 5: {
      int x, y;
      cout << "Enter X: ";
      cin >> x;
      cout << "Enter Y: ";
```

```
      cin >> y;
      cout << "Array DB: ";
      arrayDB.searchByCoordinate(x, y);
      cout << "Linked List DB: ";
      linkedDB.searchByCoordinate(x, y);
      break;
    }
    case 6: {
      string name;
      cout << "Enter name to delete: ";
      cin >> name;
      arrayDB.deleteByName(name);
      linkedDB.deleteByName(name);
      cout << "Deleted from both databases if it existed.\n";
      break;
    }
    case 7: {
      int x, y;
      cout << "Enter X: ";
      cin >> x;
      cout << "Enter Y: ";
      cin >> y;
      arrayDB.deleteByCoordinate(x, y);
      linkedDB.deleteByCoordinate(x, y);
      cout << "Deleted from both databases if it existed.\n";
      break;
    }
    case 8: {
      int x, y;
      double dist;
      cout << "Enter target X: ";
      cin >> x;
      cout << "Enter target Y: ";
      cin >> y;
      cout << "Enter maximum distance: ";
      cin >> dist;
      cout << "\n--- Array DB ---\n";
      arrayDB.printWithinDistance(x, y, dist);
      cout << "\n--- Linked List DB ---\n";
      linkedDB.printWithinDistance(x, y, dist);
      break;
    }
    default:
      cout << "Invalid Choice!\n";
    }
  }

  return 0;
}
```

OUTPUT:

```
(base) daxpatel@Daxs-MacBook-Air Linkedlist % cd /Users/daxpatel/Desktop/DSA/Linkedlist
g++ -std=c++17 unordered_list.cpp -o unordered_list
./unordered_list


========= CITY DATABASE MENU =========
1. Insert Data
2. Print List (Array DB)
3. Print List (Linked List DB)
4. Search by Name
5. Search by Coordinate
6. Delete by Name
7. Delete by Coordinate
8. Print within Distance
9. Exit
Enter choice: 1
Enter Name: Dax
Enter X: 30
Enter Y: 30
Data inserted into both databases.

========= CITY DATABASE MENU =========
1. Insert Data
2. Print List (Array DB)
3. Print List (Linked List DB)
4. Search by Name
5. Search by Coordinate
6. Delete by Name
7. Delete by Coordinate
8. Print within Distance
9. Exit
Enter choice: 2
--- Array DB ---
- Dax at (30, 30)
```

```
========= CITY DATABASE MENU =========
1. Insert Data
2. Print List (Array DB)
3. Print List (Linked List DB)
4. Search by Name
5. Search by Coordinate
6. Delete by Name
7. Delete by Coordinate
8. Print within Distance
9. Exit
Enter choice: 2
--- Array DB ---
- Dax at (30, 30)
- Dev at (15, 30)

========= CITY DATABASE MENU =========
1. Insert Data
2. Print List (Array DB)
3. Print List (Linked List DB)
4. Search by Name
5. Search by Coordinate
6. Delete by Name
7. Delete by Coordinate
8. Print within Distance
9. Exit
Enter choice: 3
--- Linked List DB ---
- Dax at (30, 30)
- Dev at (15, 30)
```

```
========= CITY DATABASE MENU =========
1. Insert Data
2. Print List (Array DB)
3. Print List (Linked List DB)
4. Search by Name
5. Search by Coordinate
6. Delete by Name
7. Delete by Coordinate
8. Print within Distance
9. Exit
Enter choice: 4
Enter name to search: Dax
Array DB: Found: Dax at (30, 30)
Linked List DB: Found: Dax at (30, 30)

========= CITY DATABASE MENU =========
1. Insert Data
2. Print List (Array DB)
3. Print List (Linked List DB)
4. Search by Name
5. Search by Coordinate
6. Delete by Name
7. Delete by Coordinate
8. Print within Distance
9. Exit
Enter choice: 5
Enter X: 15
Enter Y: 30
Array DB: Found: Dev at (15, 30)
Linked List DB: Found: Dev at (15, 30)
```

```
========= CITY DATABASE MENU =========
1. Insert Data
2. Print List (Array DB)
3. Print List (Linked List DB)
4. Search by Name
5. Search by Coordinate
6. Delete by Name
7. Delete by Coordinate
8. Print within Distance
9. Exit
Enter choice: 2
--- Array DB ---
- Dax at (30, 30)
- Dev at (15, 30)

========= CITY DATABASE MENU =========
1. Insert Data
2. Print List (Array DB)
3. Print List (Linked List DB)
4. Search by Name
5. Search by Coordinate
6. Delete by Name
7. Delete by Coordinate
8. Print within Distance
9. Exit
Enter choice: 3
--- Linked List DB ---
- Dax at (30, 30)
- Dev at (15, 30)
```

```
========= CITY DATABASE MENU =========
1. Insert Data
2. Print List (Array DB)
3. Print List (Linked List DB)
4. Search by Name
5. Search by Coordinate
6. Delete by Name
7. Delete by Coordinate
8. Print within Distance
9. Exit
Enter choice: 4
Enter name to search: Dax
Array DB: Found: Dax at (30, 30)
Linked List DB: Found: Dax at (30, 30)

========= CITY DATABASE MENU =========
1. Insert Data
2. Print List (Array DB)
3. Print List (Linked List DB)
4. Search by Name
5. Search by Coordinate
6. Delete by Name
7. Delete by Coordinate
8. Print within Distance
9. Exit
Enter choice: 5
Enter X: 15
Enter Y: 30
Array DB: Found: Dev at (15, 30)
Linked List DB: Found: Dev at (15, 30)

========= CITY DATABASE MENU =========
1. Insert Data
2. Print List (Array DB)
3. Print List (Linked List DB)
4. Search by Name
5. Search by Coordinate
6. Delete by Name
7. Delete by Coordinate
8. Print within Distance
9. Exit
Enter choice: 2
--- Array DB ---
- Dax at (30, 30)
- Dev at (15, 30)

========= CITY DATABASE MENU =========
1. Insert Data
2. Print List (Array DB)
3. Print List (Linked List DB)
4. Search by Name
5. Search by Coordinate
6. Delete by Name
7. Delete by Coordinate
8. Print within Distance
9. Exit
Enter choice: 3
--- Linked List DB ---
- Dax at (30, 30)
- Dev at (15, 30)
```

```
========= CITY DATABASE MENU =========
1. Insert Data
2. Print List (Array DB)
3. Print List (Linked List DB)
4. Search by Name
5. Search by Coordinate
6. Delete by Name
7. Delete by Coordinate
8. Print within Distance
9. Exit
Enter choice: 4
Enter name to search: Dax
Array DB: Found: Dax at (30, 30)
Linked List DB: Found: Dax at (30, 30)

========= CITY DATABASE MENU =========
1. Insert Data
2. Print List (Array DB)
3. Print List (Linked List DB)
4. Search by Name
5. Search by Coordinate
6. Delete by Name
7. Delete by Coordinate
8. Print within Distance
9. Exit
Enter choice: 5
Enter X: 15
Enter Y: 30
Array DB: Found: Dev at (15, 30)
Linked List DB: Found: Dev at (15, 30)

========= CITY DATABASE MENU =========
1. Insert Data
2. Print List (Array DB)
3. Print List (Linked List DB)
4. Search by Name
5. Search by Coordinate
6. Delete by Name
7. Delete by Coordinate
8. Print within Distance
9. Exit
Enter choice: 2
--- Array DB ---
- Dax at (30, 30)
- Dev at (15, 30)

========= CITY DATABASE MENU =========
1. Insert Data
2. Print List (Array DB)
3. Print List (Linked List DB)
4. Search by Name
5. Search by Coordinate
6. Delete by Name
7. Delete by Coordinate
8. Print within Distance
9. Exit
Enter choice: 6
Enter name to delete: Dev
Deleted from both databases if it existed.

========= CITY DATABASE MENU =========
1. Insert Data
2. Print List (Array DB)
3. Print List (Linked List DB)
4. Search by Name
5. Search by Coordinate
6. Delete by Name
7. Delete by Coordinate
8. Print within Distance
9. Exit
Enter choice: 8
Enter target X: 50
Enter target Y: 50
Enter maximum distance: 50

--- Array DB ---
Cities within distance 50 from (50, 50):
- Dax at (30, 30) [Distance: 28.2843]

--- Linked List DB ---
Cities within distance 50 from (50, 50):
- Dax at (30, 30) [Distance: 28.2843]
```

```
========= CITY DATABASE MENU =========
1. Insert Data
2. Print List (Array DB)
3. Print List (Linked List DB)
4. Search by Name
5. Search by Coordinate
6. Delete by Name
7. Delete by Coordinate
8. Print within Distance
9. Exit
Enter choice: 7
Enter X: 30
Enter Y: 1
Deleted from both databases if it existed.

========= CITY DATABASE MENU =========
1. Insert Data
2. Print List (Array DB)
3. Print List (Linked List DB)
4. Search by Name
5. Search by Coordinate
6. Delete by Name
7. Delete by Coordinate
8. Print within Distance
9. Exit
Enter choice: 7
Enter X: 15
Enter Y: 30
Deleted from both databases if it existed.
```

Department of Computer Science and Engineering,
School of Technology,
Pandit Deendayal Energy University (PDEU)
Design Analysis and Algorithm Laboratory (24CS210P)

Lab Assignment 5

Name: Dax Patel
Roll no: 24BCP269

Title: Multiply matrix using different approach.

Objective: These lab experiments cover the different operations on 1D and 2D arrays , implement strassens multiplication using divide and conquer approach

Task 1: Implement both a standard O(n^3) matrix multiplication algorithm and Strassen's matrix multiplication algorithm. How big must n be before Strassen'ste the constant factors for the runtime equations of the two algorithms. How big must n be before Strassen's algorithm becomes more efficient than the standard algorithm?

CODE:

```
#include <chrono>
#include <cmath>
#include <iomanip>
#include <iostream>
#include <random>
#include <vector>

using namespace std;
using namespace std::chrono;

typedef vector<vector<int>> Matrix;

void printMatrix(const Matrix &matrix) {
  for (const auto &row : matrix) {
    for (int val : row) {
      cout << setw(5) << val << " ";
    }
    cout << "\n";
  }
  cout << "\n";
}

Matrix add(const Matrix &A, const Matrix &B) {
  int n = A.size();
  Matrix C(n, vector<int>(n));
  for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
      C[i][j] = A[i][j] + B[i][j];
  return C;
}

Matrix subtract(const Matrix &A, const Matrix &B) {
  int n = A.size();
  Matrix C(n, vector<int>(n));
  for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
      C[i][j] = A[i][j] - B[i][j];
  return C;
}

Matrix generateRandomMatrix(int n) {
  Matrix matrix(n, vector<int>(n));
  random_device rd;
```

```
  mt19937 gen(rd());
  uniform_int_distribution<> distr(-10, 10);
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
      matrix[i][j] = distr(gen);
    }
  }
  return matrix;
}

int nextPowerOf2(int n) {
  int power = 1;
  while (power < n) {
    power *= 2;
  }
  return power;
}

Matrix padMatrix(const Matrix &A, int newSize) {
  int oldSize = A.size();
  Matrix C(newSize, vector<int>(newSize, 0));
  for (int i = 0; i < oldSize; i++) {
    for (int j = 0; j < oldSize; j++) {
      C[i][j] = A[i][j];
    }
  }
  return C;
}

Matrix standardMultiply(const Matrix &A, const Matrix &B) {
  int n = A.size();
  Matrix C(n, vector<int>(n, 0));
  for (int i = 0; i < n; i++) {
    for (int k = 0; k < n; k++) {
      for (int j = 0; j < n; j++) {
        C[i][j] += A[i][k] * B[k][j];
      }
    }
  }
  return C;
}

Matrix strassenMultiplyRecursive(const Matrix &A, const Matrix &B) {
  int n = A.size();

  if (n <= 64) {
    return standardMultiply(A, B);
  }

  int half = n / 2;
  Matrix a11(half, vector<int>(half)), a12(half, vector<int>(half)),
```

```
    a21(half, vector<int>(half)), a22(half, vector<int>(half)),
    b11(half, vector<int>(half)), b12(half, vector<int>(half)),
    b21(half, vector<int>(half)), b22(half, vector<int>(half));

for (int i = 0; i < half; i++) {
  for (int j = 0; j < half; j++) {
    a11[i][j] = A[i][j];
    a12[i][j] = A[i][j + half];
    a21[i][j] = A[i + half][j];
    a22[i][j] = A[i + half][j + half];

    b11[i][j] = B[i][j];
    b12[i][j] = B[i][j + half];
    b21[i][j] = B[i + half][j];
    b22[i][j] = B[i + half][j + half];
  }
}

Matrix S1 = subtract(b12, b22);
Matrix S2 = add(a11, a12);
Matrix S3 = add(a21, a22);
Matrix S4 = subtract(b21, b11);
Matrix S5 = add(a11, a22);
Matrix S6 = add(b11, b22);
Matrix S7 = subtract(a12, a22);
Matrix S8 = add(b21, b22);
Matrix S9 = subtract(a11, a21);
Matrix S10 = add(b11, b12);

Matrix P1 = strassenMultiplyRecursive(a11, S1);
Matrix P2 = strassenMultiplyRecursive(S2, b22);
Matrix P3 = strassenMultiplyRecursive(S3, b11);
Matrix P4 = strassenMultiplyRecursive(a22, S4);
Matrix P5 = strassenMultiplyRecursive(S5, S6);
Matrix P6 = strassenMultiplyRecursive(S7, S8);
Matrix P7 = strassenMultiplyRecursive(S9, S10);

Matrix c11 = add(subtract(add(P5, P4), P2), P6);
Matrix c12 = add(P1, P2);
Matrix c21 = add(P3, P4);
Matrix c22 = subtract(subtract(add(P5, P1), P3), P7);

Matrix C(n, vector<int>(n));
for (int i = 0; i < half; i++) {
  for (int j = 0; j < half; j++) {
    C[i][j] = c11[i][j];
    C[i][j + half] = c12[i][j];
    C[i + half][j] = c21[i][j];
    C[i + half][j + half] = c22[i][j];
  }
}
```

```
  return C;
}

Matrix strassenMultiply(const Matrix &A, const Matrix &B) {
  int n = A.size();
  int m = nextPowerOf2(n);

  Matrix A_padded = padMatrix(A, m);
  Matrix B_padded = padMatrix(B, m);

  Matrix C_padded = strassenMultiplyRecursive(A_padded, B_padded);

  Matrix C(n, vector<int>(n));
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
      C[i][j] = C_padded[i][j];
    }
  }
  return C;
}

int main() {
  while (true) {
    cout << "\n========= MATRIX MULTIPLICATION MENU =========\n";
    cout << "1. Run Automated Benchmark Suite\n";
    cout << "2. Multiply Custom Matrices\n";
    cout << "3. Exit\n";
    cout << "Enter choice: ";

    int choice;
    cin >> choice;

    if (choice == 3) {
      cout << "Exiting...\n";
      break;
    }

    switch (choice) {
    case 1: {
      vector<int> sizes = {2, 4, 8, 16, 32, 64, 128, 256, 512};
      cout << setw(8) << "\nN x N" << " | " << setw(16) << "Standard O(n^3)"
           << " | " << setw(16) << "Strassen O(n^2.81)" << " | "
           << "Faster Algorithm\n";
      cout << string(68, '-') << "\n";

      for (int n : sizes) {
        Matrix A = generateRandomMatrix(n);
        Matrix B = generateRandomMatrix(n);

        auto startStd = high_resolution_clock::now();
        Matrix CStandard = standardMultiply(A, B);
```

```
      auto endStd = high_resolution_clock::now();
      double timeStd = duration_cast<microseconds>(endStd - startStd).count();

      auto startStr = high_resolution_clock::now();
      Matrix CStrassen = strassenMultiply(A, B);
      auto endStr = high_resolution_clock::now();
      double timeStr = duration_cast<microseconds>(endStr - startStr).count();

      cout << setw(8) << n << " | " << setw(13) << timeStd << " us | "
           << setw(13) << timeStr << " us | ";

      if (timeStr < timeStd) {
        cout << "STRASSEN WINS!\n";
      } else {
        cout << "STANDARD WINS\n";
      }
    }

    cout << "\n\nNote: For exact constants C_1, C_2 in C_1*n^3 and "
            "C_2*n^2.81,\n";
    cout << "Substitute the time metrics measured at the largest size bounds "
            "(e.g. N=512) into:\n";
    cout << "C_1 = time_std / (512^3)\n";
    cout << "C_2 = time_strassen / (512^2.81)\n";
    break;
  }
  case 2: {
    int n;
    cout << "Enter the size N of the N x N matrices: ";
    cin >> n;
    if (n <= 0) {
      cout << "Invalid size.\n";
      break;
    }

    Matrix A(n, vector<int>(n));
    Matrix B(n, vector<int>(n));

    cout << "Enter elements for Matrix A (" << n << "x" << n
         << "), row by row:\n";
    for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
        cin >> A[i][j];
      }
    }

    cout << "Enter elements for Matrix B (" << n << "x" << n
         << "), row by row:\n";
    for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
        cin >> B[i][j];
```

```
        }
      }

      cout << "\n--- Result (Standard O(n^3)) ---\n";
      Matrix CStandard = standardMultiply(A, B);
      printMatrix(CStandard);

      cout << "--- Result (Strassen O(n^2.81)) ---\n";
      Matrix CStrassen = strassenMultiply(A, B);
      printMatrix(CStrassen);
      break;
    }
    default:
      cout << "Invalid Choice!\n";
    }
  }

  return 0;
}
```

Output:

```
========= MATRIX MULTIPLICATION MENU =========
1. Run Automated Benchmark Suite
2. Multiply Custom Matrices
3. Exit
Enter choice: 2
Enter the size N of the N x N matrices: 2
Enter elements for Matrix A (2x2), row by row:
1
2
3
4
Enter elements for Matrix B (2x2), row by row:
5
6
3
4

--- Result (Standard O(n^3)) ---
   11    14
   27    34

--- Result (Strassen O(n^2.81)) ---
   11    14
   27    34


========= MATRIX MULTIPLICATION MENU =========
1. Run Automated Benchmark Suite
2. Multiply Custom Matrices
3. Exit
```

Question :

find the min max maiden in the vectors.

```
#include <algorithm>
#include <ctime>
#include <cmath>
#include <iostream>
#include <limits>
#include <vector>

using namespace std;

struct MinMax {
    int min;
    int max;
};

int steps = 0;

MinMax findMinMax(const vector<int>& arr, int low, int high, int level) {
    MinMax result{};

    if (low == high) {
        result.min = result.max = arr[low];
        cout << "step " << level << ": Min: " << result.min << " Max: " <<
result.max
             << '\n';
        return result;
    }

    if (high == low + 1) {
        if (arr[low] < arr[high]) {
            result.min = arr[low];
            result.max = arr[high];
        } else {
            result.min = arr[high];
            result.max = arr[low];
        }
        cout << "step " << level << ": Min: " << result.min << " Max: " <<
result.max
             << '\n';
        return result;
    }

    int mid = low + (high - low) / 2;
    MinMax left = findMinMax(arr, low, mid, level + 1);
    MinMax right = findMinMax(arr, mid + 1, high, level + 1);
```

```
    result.min = min(left.min, right.min);
    result.max = max(left.max, right.max);

    cout << "step " << level << ": Min: " << result.min << " Max: " << result.max
<< '\n';
    return result;
}

void exponentialMinMaxDfs(const vector<int>& arr,
                          int index,
                          bool hasChosen,
                          int currentMin,
                          int currentMax,
                          MinMax& answer,
                          long long& calls) {
    calls++;

    if (index == static_cast<int>(arr.size())) {
        if (hasChosen) {
            answer.min = min(answer.min, currentMin);
            answer.max = max(answer.max, currentMax);
        }
        return;
    }

    exponentialMinMaxDfs(arr, index + 1, hasChosen, currentMin, currentMax, answer,
calls);

    if (!hasChosen) {
        exponentialMinMaxDfs(arr, index + 1, true, arr[index], arr[index], answer,
calls);
    } else {
        exponentialMinMaxDfs(
            arr,
            index + 1,
            true,
            min(currentMin, arr[index]),
            max(currentMax, arr[index]),
            answer,
            calls
        );
    }
}

MinMax exponentialMinMax(const vector<int>& arr, long long& calls) {
    MinMax answer{numeric_limits<int>::max(), numeric_limits<int>::min()};
    calls = 0;
    exponentialMinMaxDfs(arr, 0, false, 0, 0, answer, calls);
    return answer;
}
```

```cpp
int partition(vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low;

    for (int j = low; j < high; j++) {
        steps++;
        if (arr[j] <= pivot) {
            swap(arr[i], arr[j]);
            i++;
        }
    }

    swap(arr[i], arr[high]);
    return i;
}

int quickSelect(vector<int>& arr, int low, int high, int k) {
    if (low <= high) {
        int pi = partition(arr, low, high);
        if (pi == k) {
            return arr[pi];
        } else if (pi > k) {
            return quickSelect(arr, low, pi - 1, k);
        } else {
            return quickSelect(arr, pi + 1, high, k);
        }
    }
    return -1;
}

double findMedian(vector<int> arr) {
    int n = static_cast<int>(arr.size());
    if (n == 0) {
        return 0.0;
    }

    if (n % 2 == 1) {
        return quickSelect(arr, 0, n - 1, n / 2);
    }

    int val1 = quickSelect(arr, 0, n - 1, n / 2);
    int val2 = quickSelect(arr, 0, n - 1, n / 2 - 1);
    return (val1 + val2) / 2.0;
}

int main() {
    int m;
    cout << "Enter the number of elements: ";
    cin >> m;

    if (m <= 0) {
```

```
        cout << "Please enter a positive size.\n";
        return 0;
    }

    vector<int> arr(m);
    for (int i = 0; i < m; i++) {
        cout << "Enter element " << i + 1 << ": ";
        cin >> arr[i];
    }

    int n = static_cast<int>(arr.size());

    clock_t start1 = clock();
    MinMax ans = findMinMax(arr, 0, n - 1, 0);
    clock_t end1 = clock();

    clock_t start2 = clock();
    double median = findMedian(arr);
    clock_t end2 = clock();

    bool ranExponential = false;
    MinMax exponentialAns{};
    long long exponentialCalls = 0;
    double time_exponential_ms = 0.0;

    if (n <= 20) {
        ranExponential = true;
        clock_t start3 = clock();
        exponentialAns = exponentialMinMax(arr, exponentialCalls);
        clock_t end3 = clock();
        time_exponential_ms = 1000.0 * static_cast<double>(end3 - start3) /
CLOCKS_PER_SEC;
    }

    cout << "Final Min = " << ans.min << '\n';
    cout << "Final Max = " << ans.max << '\n';
    cout << "Median = " << median << '\n';
    cout << "Total comparison steps: " << steps << '\n';

    if (ranExponential) {
        cout << "Exponential Min = " << exponentialAns.min << '\n';
        cout << "Exponential Max = " << exponentialAns.max << '\n';
        cout << "Exponential recursion calls: " << exponentialCalls << '\n';
    } else {
        cout << "Exponential min-max skipped for n > 20 (too slow for O(2^n)).\n";
    }

    double time_min_max_ms = 1000.0 * static_cast<double>(end1 - start1) /
CLOCKS_PER_SEC;
    double time_median_ms = 1000.0 * static_cast<double>(end2 - start2) /
CLOCKS_PER_SEC;
```

```
    cout << "Execution Time (min-max): " << time_min_max_ms << " ms\n";
    cout << "Execution Time (median): " << time_median_ms << " ms\n";
    if (ranExponential) {
        cout << "Execution Time (exponential min-max): " << time_exponential_ms <<
" ms\n";
    }

    return 0;
}
```

Output:

```
(base) daxpatel@Daxs-MacBook-Air LAB5 % cd "/Users/daxpatel/Desktop/Sem-4/DAA/LAB/LAB5/" && g++ min_max.cpp -o min_
Enter the number of elements: 8
Enter element 1: 2
Enter element 2: 64
Enter element 3: 12
Enter element 4: 53
Enter element 5: 6
Enter element 6: 8
Enter element 7: 2
Enter element 8: 3
step 2: Min: 2 Max: 64
step 2: Min: 12 Max: 53
step 1: Min: 2 Max: 64
step 2: Min: 6 Max: 8
step 2: Min: 2 Max: 3
step 1: Min: 2 Max: 8
step 0: Min: 2 Max: 64
Final Min = 2
Final Max = 64
Median = 7
Total comparison steps: 31
Exponential Min = 2
Exponential Max = 64
Exponential recursion calls: 511
Execution Time (min-max): 0.061 ms
Execution Time (median): 0.021 ms
Execution Time (exponential min-max): 0.019 ms
(base) daxpatel@Daxs-MacBook-Air LAB5 %
```

Question: Karatsuba

CODE:

```
#include <iostream>
#include <ctime>

using namespace std;

long long power10(int n) {
    long long result = 1;
    for (int i = 0; i < n; i++)
        result *= 10;
    return result;
}
```

```
int numDigits(long long x) {
    int count = 0;
    if (x == 0) return 1;
    while (x != 0) {
        count++;
        x /= 10;
    }
    return count;
}


long long normalMultiply(long long x, long long y) {
    return x * y;
}

long long karatsuba(long long x, long long y) {
    if (x < 10 || y < 10)
        return x * y;

    int n1 = numDigits(x);
    int n2 = numDigits(y);
    int n = (n1 > n2) ? n1 : n2;
    int half = n / 2;

    long long high1 = x / power10(half);
    long long low1  = x % power10(half);
    long long high2 = y / power10(half);
    long long low2  = y % power10(half);

    long long z0 = karatsuba(low1, low2);
    long long z1 = karatsuba((low1 + high1), (low2 + high2));
    long long z2 = karatsuba(high1, high2);

    return z2 * power10(2 * half) + (z1 - z2 - z0) * power10(half) + z0;
}

int main() {
    long long x, y;

    cout<<"Enter first number:"<<endl;
    cin>>x;

    cout<<"Enter second number:"<<endl;
    cin>>y;


    clock_t start1 = clock();
    long long result1 = normalMultiply(x, y);
    clock_t end1 = clock();
```

```
    clock_t start2 = clock();
    long long result2 = karatsuba(x, y);
    clock_t end2 = clock();

    double time_normal = ((double)(end1 - start1)) / CLOCKS_PER_SEC * 1000;
    double time_karatsuba = ((double)(end2 - start2)) / CLOCKS_PER_SEC * 1000;

    cout<<"Result using Normal Multiplication:"<<result1<<endl;
    cout<<"Result using Karatsuba Multiplication:"<<result2<<endl;

    cout<<"Execution Time (Normal):" <<time_normal<<" ms"<<endl;
    cout<<"Execution Time (Karatsuba):"<< time_karatsuba<<" ms";

    return 0;
}
```

OUTPUT:

```
(base) daxpatel@Daxs-MacBook-Air LAB % cd "/Users/daxpatel/Desktop/Sem-4/DAA/LAB/LAB5/" && g++ karatsuba.cpp -o karatsuba &&
suba
Enter first number:
6
Enter second number:
32
Result using Normal Multiplication:192
Result using Karatsuba Multiplication:192
Execution Time (Normal):0.004 ms
Execution Time (Karatsuba):0 ms%
(base) daxpatel@Daxs-MacBook-Air LAB5 %
```